Reproducing “Near-Duplicate Detection for Web Crawling”

Name Surname

2026-08-01

Abstract This paper aims to reproduce the findings of the “Near-Duplicate Detection for Web Crawling” work, using PySpark to scale the proposed algorithm, under different datasets, feature models and parameters.

Introduction

This work aims to answer the following question: given two datasets, is it possible to detect near duplicate pairs of data efficiently?

That question was tackled by Manku et al.'s paper [1], applied to web crawling.

The “Definitions and Terminology” chapter formalizes and explains the terms used throughout this paper, while the “Methodology” chapter contains the higher-level logic of the solution proposed. “Implementation” chapter talks about implementation details and lastly “Result” shows an application of the implementation across multiple datasets.

Definitions and Terminology

This paragraphs includes common terminology used in later chapters. In this paper only textual data (i.e. strings) are considered, however the approach is similar to any other type of data.

- Two data are *duplicate* if they are exactly equal. In practice, their hash digest is the same.
- Two data are *near-duplicate* if they are *similar enough* to some criteria.
- A *feature* is a property of some data, obtained from the application of a *feature model*. For example, a feature value can be 32 and its feature model might be “the number of words in the string”. Note that a single data might have multiple features.
- A *fingerprint* is a binary string of fixed length that represents the content of some data.
- A *partition* of a fingerprint is a substring of that fingerprint. Each partition is exclusive: it cannot overlap with other partitions.
- A *schema* defines how many partitions should be used and how to combine them for the near-duplicate algorithm.

Methodology

Consider an easier variant of the tracted problem: given two datasets, is it possible to detect ~~near~~ duplicate pairs of data ~~efficiently~~?

This can be trivially solved computing hash digests for every data in the datasets and using a Cartesian product: if two digests match then they are duplicate (ignoring hash collisions).

However, this wouldn't work to detect *near* duplicates: two almost identical data might have wildly different digests. Also, a Cartesian product is too inefficient to process large datasets.

The following sub-chapters aim to solve these issues, replicating the original paper methodology.

1. Fingerprint Generation

As previously mentioned, hash functions can't represent data for near-duplicate detection purposes.

In order to solve this issue, Charikar's SimHash [2] algorithm is used. Given an input string and a feature model, its approach can be described as follows: 1. Features are extracted from the input; 2. All features are hashed; 3. The features digests are combined into a fingerprint, taking the most popular bit value for each bit index, across all digests.

The objective of this first phase is to transform each string of a dataset into a single, fixed-length binary string, referred to as a *fingerprint*. A *fingerprint* can be considered as the "identifier" of that string, that represents "what it means". This operation is usually done once, however some feature models require to perform this phase again when the dataset changes.

The process to compute fingerprints starts with the extraction of features from all strings in the dataset, following the chosen feature model. Now each string has a set of features, called the *feature set*. Then all features are hashed, using an hashing function of choice, obtaining the digests, a set of binary values with fixed length. Lastly, all feature digests that belong to some string are reduced to a single binary value (a *fingerprint*) with the following criteria: for all bit indexes, the fingerprint has value 1 if most digests have value 1 in that bit index, 0 otherwise. This algorihtm is called SimHash [2], presented by Charikar et al.

Selecting the appropriate feature model is crucial. The original paper proposes various feature models, such as *Shingles* and *TF-IDF*. In this work, four feature models are tested and compared: *Bag of Words*, *Shingles*, *TF-IDF* and *Embeddings*.

Bag of Words

For a given text, its features are the individual words it contains. Hence, two texts are considered near-duplicate if they contain about the same words. This approach is computationally efficient and simple to implement, but it fails to preserve spatial locality. For example, the sentence "The bank is after the office" yields the exact same features as "The office is after the bank".

Shingles

Generally, a shingle is a contiguous sequence of n items from a data source; in this context, it refers to an overlapping contiguous sequence of n characters. Shingles are generated using a sliding window algorithm with a window size of n. This model preserves locality to some extent, but it generates a large number of features, which can negatively impact performance. To reduce the feature count, a non-overlapping variant is considered. Additionally, word-based shingles (as opposed to traditional character-based ones) are evaluated.

TF-IDF

This is a refined version of *Bag of Words* that accounts for both the frequency of a word within a specific text (DF) and its rarity across the entire dataset (IDF):

$$\text{TF-IDF}(term, document, Dataset) = TF(term, document) * IDF(term, Dataset)$$

TF stands for Term Frequency, which assigns higher importance to words that appear frequently in a given text:

$$TF(term, document) = \frac{\text{How many times term appears in document}}{\text{How many words document has}}$$

IDF stands for Inverse Document Frequency, which assigns higher weight to words that appear in fewer distinct documents. This filters out very common words, such as conjunctions and verbs:

$$IDF(term, Dataset) = \log(\frac{\text{Total number of documents in Dataset}}{\text{Number of documents that contain term}})$$

This is the feature model chosen in the original paper [1]. Despite its higher accuracy, it's computationally more expensive than previous models, which were "local" in the sense that computing the features of a single text did not require information about the entire dataset. Furthermore, maintenance introduces additional overhead, as the weights of each term must be recomputed whenever the dataset changes.

Embeddings

An embedding model is a machine learning algorithm that maps text into a vector capturing its semantic meaning. In this study, a transformer-based model, *Qwen3-embedding-0.6b* [3], is used and executed locally using Ollama. Each weight of the generated embeddings is quantized to 0/1, taking the median of all weights as pivot, becoming a bit of the result fingerprint. Although suboptimal, this is required in order to build binary fingerprints. While all previous models rely on syntax to extract features, embedding models operate on semantics, allowing for the detection of semantical near-duplicate text.

2. Fingerprint Matching

After the previous section, each string now has an identifying fingerprint. The goal for this phase is to compare fingerprints from two datasets, called *dataset* and *query* for simplicity sake, in order to find pairs of fingerprints whose Hamming distance is less than K, a fixed integer value. The reference paper [1] approaches this problem by splitting each fingerprint into at least $K + n$ *partitions*, for $n > 0$. This grants a useful property: if two fingerprints are near-duplicate by a factor of K, then at least n of their $K + n$ partitions must be equal (the opposite doesn't hold - other partitions might have more than K different bits). The optimal schema is derived by the following formula:

$$X(f, k, d) = \begin{cases} 1 & \text{if } \ d < \tau \\ \min_{r>k}\binom{r}{k} \cdot X\left(\frac{fk}{r}, k, d - \frac{(r-k)f}{r}\right) & \text{otherwise} \end{cases}$$

where f is the size of each fingerprint, K is the maximum allowed Hamming distance, d is the log_2 size of query and $p_m in$ is the maximum allowed candidate set size (in log_2).

Manku's approach can be summerized as follows:

1. n copies of *query* are generated, where $n = \binom{r}{k}$, computed by the previous formula
2. For each copy, a schema instance is applied: a partition of all fingerprints of a copy are cut and pasted on the head of all fingerprints
3. Fingerprints are sorted and then compressed, using Huffman codes

4. All query copies are broadcasted to workers
5. Workers iterate over *dataset*, extracting partitions from all rows and checking if it matches at least one of the *query* fingerprints (with the same schema instance), using binary search
6. Fingerprint pairs with matching partitions *might* be near-duplicate: if their Hamming distance is less than K they are considered a near-duplicate pair

Implementation

This implementation uses the PySpark [4] framework, which allows to trivially scale the computation to clusters, for large datasets. The next paragraphs will include notes and observations about the PySpark implementation.

Feature Models

The implemented feature models are *Bag Of Words*, *Shingles*, *TF-IDF* and *Embeddings*. All of them are based on PySpark's DataFrames, using native methods from the package `pyspark.sql.functions`. When necessary, Pandas UDF are adopted.

Bag of Words and Shingles

Their implementation is straightforward.

TF-IDF

In PySpark, this model is implemented using `pyspark.ml.features.IDF` and `pyspark.ml.features.CountVectorizer`. A custom implementation was developed and scrapped: its performance were much worse than the methods provided by the PySpark library.

Embeddings

In this study, a transformer-based model, *Qwen3-embedding-0.6b* [3], is used and executed locally using Ollama. Despite being a small model, its memory requirements can be quite taxing for large inputs: it's recommended to run this model on machines with at least 12GB to 16GB of RAM. Unfortunately, all benchmarks on this paper are executed on a 8GB machine, which required some compromises.

FastSimHash

SimHash can be naively implemented using an array of counters and iterating over all feature's digests. It's possible to optimize it by using Numba, a Just-In-Time compiler for Python that translates Python and NumPy code into machine code, using SIMD instructions wherever possible. Instead of iterating over all digests, it iterates over byte indexes and uses 8 counters (one for each bit of a byte) to reduce memory updates. This is inspired by Otmar Ertl's FastSimHash implementation [5], which uses a SWAR (SIMD Within A Register) technique instead of pure SIMD instructions. It's a remarkable improvement: performing SimHash on a laptop with 1000000 digests takes around 14000ms, while FastSimHash takes around 400ms (including compilation time, on a cold boot).

Fingerprint Detection with RDDs

The detection algorithm from the original paper [1] has been replicated in PySpark using RDDs, with some differences. Instead of using a binary search algorithm, an hash map is used: it requires slightly more memory, but it optimizes the candidate search from $O(logn)$ to $O(1)$. The Huffman Code compression strategy is not implemented: it would require a fixed K value, which is not the case for this study. However, it is certainly recommended for production environments where fingerprints are stored, with a fixed K value.

Results

qqp dataset

This dataset is the de-facto standard for near-duplicate detection algorithms. It features more than 300,000 rows of labelled text pairs, which are titles extracted from Quora's questions, a rather famous website. In this benchmark, a sample of 199 rows are considered, splitted in 76 near-duplicates and 123 non near-duplicates. [put reason for such small sample]

[image-1] shows feature model's f1-score on the chosen sample: as expected, Embeddings is the best performing model, although not by much. [image-2] shows a clearer image: Embeddings is the only model capable of truly detecting duplicates, as shown by the spike around $K = 6$.

However, Embeddings score isn't as high as one might expect: it's possible that the heavy quantization applied to the produced weights drags down its score, or the (rather small) embedding model might not be "smart" enough for this task. In order to verify this, the computed f1-score is compared to the result of Gill et al. [source]. Literature shows that larger, state-of-the-art generic-purpose embedding models settle on a f1-score of 0.77, which isn't far from the best obtained score from this work (which is 0.68). Moreover, Gill et al. show that fine-tuning can drastically improve performance: in this work, the embedding model is used as-is. This can be an idea for a point of improvement. [TODO: compute best score with embeddings on K=4, 5 or 6 - it might be higher than 0.68, on the whole QQP]

From [image-2] it's possible to assume that larger fingerprint sizes don't affect the f1-score (for this dataset). Most likely, large fingerprint sizes might be useful in huge datasets, where collision risk is higher.

It's important to also consider the optimal K value: if the optimal K value is too high then this algorithm isn't optimal. As shown in chapter [link-to-chapter-with-K-complexity-graph], the K value must be low enough to guarantee decent performance. If this wasn't the case, a trivial Cartesian product would be more efficient. Considering the Embedding feature model and a fingerprint size of 64 bits, the optimal K value seems to be between 3 to 6, from image [image-3].

Feature models: efficiency

In this section, feature models's efficiency is tested. As [...] shows, all feature models take about the same time, except for TF-IDF. The other models are all local: each data elaboration is independent from other, which makes it efficient and easy to compute across workers. On the other hand, TF-IDF cannot replicate this: it's necessary to create a large, global data structure that counts how many times each word appears in all the dataset, resulting in a slower computation.

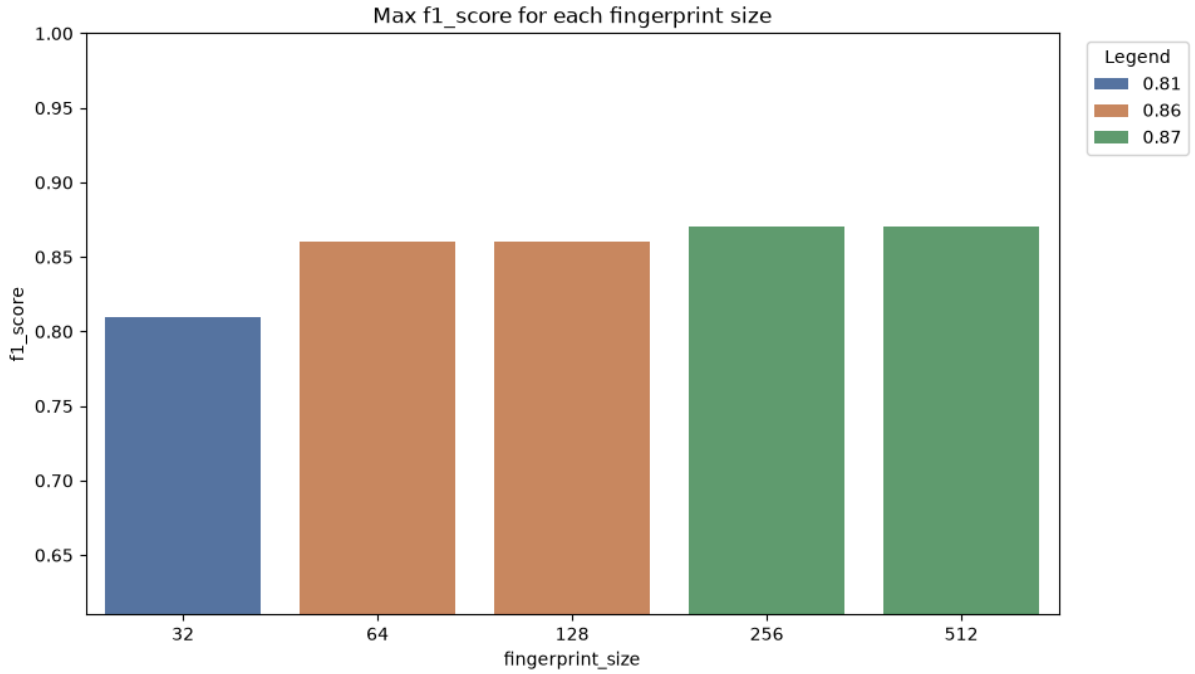
1. Fingerprint Generation

The proposed feature models are compared across two datasets: *llm-paraphrases* and *wikipedia*. Each row of these datasets contains two text columns and a source of truth column, which allows to compare how good is a certain feature model to another. The former dataset is semantic - two texts are considered near-duplicate iff they mean the same thing. The latter is syntactical - rows considered near-duplicate contain the same text, but with around 10% of noise (ie, altered words).

Semantic Duplicate Detection

The semantic duplicate detection benchmark is performed on the llm-paraphrases dataset. In this case the embeddings model reaches the highest score, followed by TF-IDF and bag_of_words. This is within expectations: despite its small size, the embedding model outperforms classic

NLP feature model on semantic similarities. Fingerprint sizes of at least 64 bits perform about the same, while 32 bits doesn't seem to be enough to represent features in this case. ...



Syntactical Duplicate Detection

The syntactical duplicate detection benchmark is based on the wikipedia dataset [6], modified for this task. Each row contains two wikipedia entries - the second entry can be a copy of the first one with about 10% of syntax noise, or talk about something completely different. All feature models achieve perfect scores, even on the smallest fingerprint size. ...

Fingerprint Matching Performance

As shown in ..., a dataset of $1,000,000$ 64-bit fingerprints, splitted between corpus and query in a 80:20 ratio, takes about ... seconds to compute all near-duplicates with $K = 3$. This is ran on a MacBook Air M1 laptop. Higher values of K significantly increase the required time, as expected: with the same environment, raising K to 5 takes ... seconds.

Conclusion

The results have shown that the near-duplicate detection algorithm ideated by Manku et al [1] is consistent with literature. Also, it's flexible enough to support advanced feature models like embeddings, detecting semantic near-duplicates for low K values.

Bibliography

[1] A. D. S. Gurmeet Singh Manku Arvind Jain, "Detecting Near-Duplicates for Web Crawling," 2007.

[2] M. S. Charikar, "Similarity Estimation Techniques from Rounding Algorithms," 2002.

[3] Y. Zhang *et al.*, "Qwen3 Embedding: Advancing Text Embedding and Reranking Through Foundation Models," *arXiv preprint arXiv:2506.05176*, 2025.

[4] [Online]. Available: https://spark.apache.org/docs/latest/api/python/index.html

[5] O. Ertl, “Speeding up SimHash by 10x using a bit hack,” 2023.

[6] W. Foundation, “Wikimedia Downloads.” [Online]. Available: https://dumps.wikimedia.org/